Employee Attendance & Payroll Automation system

WEEK 03 TASK

Technical Architecture & Database Selection Report

Name: ANOUSHEY CHANDIO

Position: Python Development Intern

Internship Program: 1-Month Internship Program (Batch 02)

Company: Aptura Tech Solutions

Date: August 2026

Question 01:

Assume your application is used by 500 companies, each with over 10,000 employees. Explain how you would redesign the application architecture to improve scalability, maintainability, and performance.

Answer:

- To scale the application for 500 companies and 5 million users, I would redesign the system using these key steps:
- **Database Indexing & Isolation:**
- I will add strict indexing on company_id to separate company data. This keeps search queries fast even with millions of records.
- **Database Connection Pooling & Sharding:**
- I will use connection pooling (like PgBouncer) to handle peak morning check-ins. If traffic grows further, we can shard the database by company_id.
- **Caching Layer (Redis):**
- I will store active user sessions and shift statuses in Redis. This reduces direct database hits during high-traffic hours.
- **Background Task Queue:**
- Heavy operations like bulk payroll calculations and PDF report exports will run asynchronously in the background using Celery and Redis.
- **Stateless Backend:**

- I will keep the application backend stateless behind a Load Balancer so we can easily add more servers as traffic increases.

Question 02:

Identify at least five possible failure scenarios (e.g., corrupted files, invalid input, concurrent access, system crash, network interruption). Explain how your application will recover from each scenario while preventing data loss.

Answer:

- **Network Interruption:**

  Solution: I will use unique transaction tokens on API endpoints. If the internet drops and retries, the server recognizes the token and avoids duplicate check-in entries.

- **Concurrent Access (Race Conditions):**

  Solution: I will implement Database Row-Level Locking (FOR UPDATE) during transactions so simultaneous requests from multiple tabs/devices are processed one by one.

- **System Crash During Payroll Processing:**

  Solution: I will process payroll in micro-batches and track their status in the database. If the server crashes, it will automatically resume from the exact batch where it stopped.

- **Corrupted File Uploads / Invalid Data:**

  Solution: I will use Pydantic and Pandas for strict input validation. The database write will run inside an atomic transaction, triggering a complete ROLLBACK if any row fails validation.

- **Database Failure or Downtime:**

  Solution: I will set up automated database replication along with Point-in-Time Recovery (PITR) and regular backups to restore data quickly without loss.
